📷 Computer Vision Lab — Sampling & Image Quality

Q: A camera captures an image that appears blurred due to improper sampling. Explain how sampling affects image quality and suggest a method to correct it.

1. What is Sampling?

Sampling is the process of converting a **continuous** (real-world) image into a **digital** image by picking pixel values at regular intervals.

🌍 Continuous Image (Real World) → 📷 Sampling (Pick Pixels) → 🖼️ Digital Image (Pixels)

✅ **High Sampling Rate** → More pixels → Better image quality → More details

❌ **Low Sampling Rate** → Fewer pixels → Loss of details → Blurred image

2. Why Does Blurring Occur?

When the sampling rate is too low:

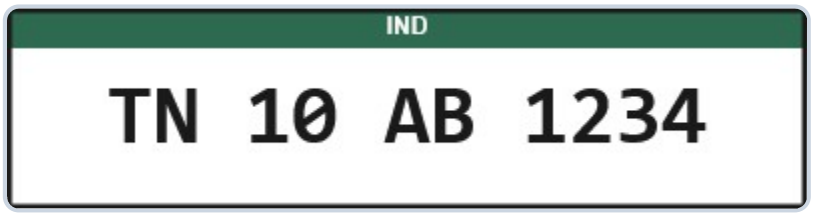
- Fine image details are **lost**
- Neighboring pixels **merge together**
- Edges become **unclear**
- Small objects **disappear**
- The image looks **blurred or blocky**

💡 **Key Point:** More pixels capture more details. Better sampling = Better image quality.

3. Example: Vehicle Number Plate

See how different sampling rates affect a number plate image:

Sampling Rate: 100%



400×100	Sharp	99%	TN 10 AB 1234
Resolution	Quality	OCR Accuracy	OCR Result

Sampling Rate	Resolution (W × H)	Quality	OCR Accuracy
Low	50 × 25 pixels	Blurred	45%
Medium	200 × 100 pixels	Average	82%
High	400 × 200 pixels	Sharp	99%

4. How to Correct Blurring?

1 📷 **Increase Camera Resolution**

Use a higher-resolution camera. Example: Instead of 640×480, use 1920×1080.

2 🔍 **Increase Sampling Rate**

Capture more pixels per unit area. More pixels → Better detail → Clear image.

3 🖥️ **Use Image Interpolation (Software)**

Resize using: **Bilinear**, **Bicubic**, or **Lanczos** interpolation. *(Improves appearance but cannot recover lost details.)*

4 🎯 **Improve Camera Focus**

Proper focus ensures the sampled pixels accurately represent the scene.

5. Real-Life Applications Affected by Poor Sampling

- **Face Recognition** — Face cannot be identified
- **Medical Imaging** — Tumors may be missed
- **Self-Driving Cars** — Road signs not detected
- **QR Scanner** — QR code cannot be decoded
- **OCR** — Text recognition accuracy drops

6. Python Code — Try It Yourself

Use these OpenCV code snippets in your lab to experiment with sampling and interpolation.

📷 Required Image for Code

Download the sample number plate image to run the code below.

⬇️ Download Image

🚀 How to Run this Code and See the Output

Step 1: Where to run it?

- **Google Colab / Jupyter Notebook (Recommended for Labs):** Create a new notebook. Upload the `number_plate.jpg` file into the Colab files section. Paste the code into a cell and press Shift+Enter. The output images will appear directly below the cell!
- **Local Python Script (.py):** Create a file named `lab_demo.py` in the exact same folder as the downloaded image. Paste the code into the file, open your terminal/command prompt, and run `python lab_demo.py`.

Step 2: Install Libraries (Only if running locally)

```
pip install opencv-python matplotlib numpy
```

📋 Copy

Step 3: How to see the output?

- The code uses `plt.show()` which triggers the output visualization.
- **In Colab/Jupyter:** The image will instantly render inside the webpage.
- **Running Locally:** A new desktop window will pop up showing the result. *Note: You must close the popup window for the script to continue running!*

6.1 Read and Display an Image

Copy

```
import cv2                                    # Import OpenCV library for image proc
import matplotlib.pyplot as plt               # Import Matplotlib for displaying ima

# Read the image from the disk into a NumPy array
img = cv2.imread('number_plate.jpg')          # cv2.imread reads the image in BGR (I
# Convert the BGR image to RGB format since Matplotlib expects RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # This ensures colors look correct whe

# Display the image using Matplotlib
plt.imshow(img_rgb)                           # Render the RGB image array
plt.title('Original Image')                   # Add a title to the plot window
plt.axis('off')                               # Hide the X and Y axes (ticks and bo
plt.show()                                    # Show the actual window on screen

# Print the dimensions of the loaded image
print("Image Shape:", img.shape)              # Outputs (height, width, channels) e
```

6.2 Simulate Low Sampling (Downsample)

Copy

```
# Downsample the image to a tiny 50x25 resolution to simulate a very low sampling rat
low_res = cv2.resize(img, (50, 25), interpolation=cv2.INTER_NEAREST)              #
print("Low-Res Shape:", low_res.shape)        # The image is now only 25 pixels high

# Scale the tiny image back up to 400x200 so we can actually see the blocky pixelatio
pixelated = cv2.resize(low_res, (400, 200), interpolation=cv2.INTER_NEAREST) # Upscal

# Create a figure with a specific size (width=10, height=4 inches)
plt.figure(figsize=(10, 4))                                                  # Create a vis
```

```
# Create the left subplot (1 row, 2 columns, 1st position)
plt.subplot(1, 2, 1)                                                    # Position th
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # Display the original, high-qualit
plt.title('Original (High Sampling)')                                   # Set a descr
plt.axis('off')                                                         # Hide axis t

# Create the right subplot (1 row, 2 columns, 2nd position)
plt.subplot(1, 2, 2)                                                    # Position th
plt.imshow(cv2.cvtColor(pixelated, cv2.COLOR_BGR2RGB)) # Display the blurred/blocky
plt.title('Pixelated (Low Sampling)')                                   # Set a descr
plt.axis('off')                                                         # Hide axis t
plt.show()                                      # Render both subplots to the screen
```

6.3 Compare Different Sampling Rates

Copy

```
# Define a list of different resolutions to test (Low, Medium, High)
sizes = [(50, 25), (200, 100), (400, 200)]     # Width x Height tuples
# Labels for our plot titles
labels = ['Low (50x25)', 'Medium (200x100)', 'High (400x200)']          # Define corr

plt.figure(figsize=(14, 4))                    # Create a wide figure window

# Loop through each size and label simultaneously
for i, (size, label) in enumerate(zip(sizes, labels)):                  # Iterate thr
    # Step 1: Downsample the original image to the specific 'size' to simulate the s
    small = cv2.resize(img, size, interpolation=cv2.INTER_AREA)

    # Step 2: Upsample it back to 400x200 so we can compare them all at the same phy
    restored = cv2.resize(small, (400, 200), interpolation=cv2.INTER_NEAREST)  # Ups

    plt.subplot(1, 3, i + 1)                   # Create a subplot (1 row, 3 columns,
    plt.imshow(cv2.cvtColor(restored, cv2.COLOR_BGR2RGB)) # Show the image
    plt.title(label)                           # Set the title to the current sampli
    plt.axis('off')                            # Hide the axis

plt.suptitle('Effect of Different Sampling Rates') # Add a main title for the whole
plt.tight_layout()                             # Automatically adjust spacing betwee
plt.show()                                     # Display the final comparison
```

6.4 Correction Using Interpolation Methods

Copy

```
# Start with a heavily downsampled (blurred) low-resolution image
low_res = cv2.resize(img, (50, 25), interpolation=cv2.INTER_AREA)      # Apply resize

# Define a dictionary of the 4 mathematical interpolation methods OpenCV provides fo
methods = {                                                            # Map names t
    'Nearest Neighbor': cv2.INTER_NEAREST,     # Fast, but creates blocky pixelated
    'Bilinear': cv2.INTER_LINEAR,              # Uses 4 nearby pixels, creates smoot
    'Bicubic': cv2.INTER_CUBIC,                # Uses 16 nearby pixels, creates shar
    'Lanczos': cv2.INTER_LANCZOS4              # Uses math formulas over 8x8 pixels,
}

plt.figure(figsize=(14, 4))                    # Create a wide figure to fit 4 image

# Loop through our dictionary of methods
for i, (name, method) in enumerate(methods.items()):                   # Iterate thr
    # Here is the correction! Upscale the low-res image using the current mathematic
    upscaled = cv2.resize(low_res, (400, 200), interpolation=method)

    plt.subplot(1, 4, i + 1)                   # Place image in a 1x4 grid
    plt.imshow(cv2.cvtColor(upscaled, cv2.COLOR_BGR2RGB)) # Convert BGR to RGB and s
    plt.title(name, fontsize=9)                # Label the image with the method nam
    plt.axis('off')                                                    # Hide axis t

plt.suptitle('Interpolation Methods Comparison')                       # Set a maste
plt.tight_layout()                                                     # Automatical
plt.show()                                 # Display the corrected images side-b
```

6.5 Measure Image Quality (PSNR)

Copy

```
# PSNR = Peak Signal-to-Noise Ratio. It mathematically compares a degraded image to
original = cv2.resize(img, (400, 200))         # Set our baseline ground truth image

# Test 3 different sampling sizes
for label, size in zip(['Low', 'Medium', 'High'], [(50,25), (200,100), (400,200)]):
    small = cv2.resize(img, size, interpolation=cv2.INTER_AREA)              # Simula
    restored = cv2.resize(small, (400, 200), interpolation=cv2.INTER_CUBIC) # Correc

    # Calculate the mathematical quality score between the original and the restored
    psnr = cv2.PSNR(original, restored)                                    # Compute Pea
```

```
    print(f"{label} Sampling -> PSNR: {psnr:.2f} dB") # Print the result (e.g., "Low
```

> 💡 **Output Example:** Low → ~20 dB (poor), Medium → ~30 dB (okay), High → ~∞ dB (identical)

📌 Conclusion

Improper sampling captures fewer pixels, causing **blurred images** and loss of important details. Increasing the sampling rate (higher resolution) and using proper image acquisition techniques produces **clearer images**, improving the accuracy of computer vision tasks.

Computer Vision Lab · Saveetha School of Engineering

This page/site is developed and maintained by Dr. T. M. Usha